

RAG Integration Project

Retrieval-Augmented Generation for
the Webstack Chat Application

Built with Claude Code | February 23, 2026



1. Project Overview

This project adds Retrieval-Augmented Generation (RAG) capabilities to an existing production webstack running on a Mac Mini. The webstack serves a chat interface powered by LM Studio (a local LLM server) through a Docker Compose stack of Caddy, Node.js/Express, and ChromaDB, accessible via Cloudflare.

The RAG system allows users to upload documents (PDF, DOCX, CSV, TXT), which are chunked, embedded, and stored in a ChromaDB vector database. When the RAG toggle is enabled in the chat interface, user queries are matched against stored document chunks and the relevant context is injected into the LLM prompt, enabling accurate answers grounded in the uploaded content.

Goals

- Enable document-grounded Q&A in the existing chat interface
- Support multiple document formats: PDF, Word, CSV, and plain text
- Provide a dedicated admin page for document upload and management
- Preserve all existing functionality (chat, web search, streaming, memory)
- Run entirely locally with no external API dependencies for embeddings

2. System Architecture

Infrastructure (Docker Compose)

The system runs three containers orchestrated by Docker Compose:

Service	Image	Purpose
caddy	caddy:latest	Reverse proxy, SSL termination (port 80)
app (nodeapp)	node:20-slim	Express server, RAG pipeline, chat API
chromadb	chromadb/chroma	Vector database for document embeddings

The node:20-slim image (Debian-based) is required because the embedding library (onnxruntime) needs glibc, which Alpine Linux does not provide. ChromaDB runs as a separate service because the JS client is HTTP-only and cannot run in-process.

LM Studio runs directly on the Mac host (not in Docker) and is accessible from containers via host.docker.internal:1234. Cloudflare handles external SSL termination and proxies traffic to Caddy on port 80.

Data Flow: Document Ingestion

When a user uploads a document through the /rag-admin page:

- 1. The file is uploaded to the Express server via multipart form POST
- 2. A format-specific extractor converts the file to plain text
- 3. The text is split into overlapping chunks (500 chars, 50 char overlap)
- 4. Each chunk is embedded using all-MiniLM-L6-v2 (384-dimensional vectors)
- 5. Chunks, embeddings, and metadata are stored in ChromaDB

Data Flow: RAG Query

When a user sends a chat message with the RAG toggle enabled:

- 1. The user's query is embedded using the same model
- 2. ChromaDB performs cosine similarity search (top 4 results)
- 3. Matching chunks are formatted into a context block with source citations
- 4. The context is appended to the system prompt sent to LM Studio
- 5. The LLM generates a response grounded in the retrieved documents

3. RAG Module (app/rag/)

The RAG logic is organized as a self-contained module under app/rag/ with five files:

embeddings.js

Handles local text embedding using @xenova/transformers, which runs the all-MiniLM-L6-v2 model via ONNX Runtime directly in Node.js. The model (~25MB) is downloaded on first use and cached in a persistent Docker volume (hf_cache).

Exports: initEmbeddings(), embedText(string), embedBatch(string[])

vectorstore.js

Manages the ChromaDB connection and collection operations. Connects to the ChromaDB container via HTTP (http://chromadb:8000). Uses pre-computed embeddings rather than relying on ChromaDB's built-in embedding functions, avoiding native dependency issues.

Exports: initChroma(), addDocuments(), queryByEmbedding(), deleteByDocumentId(), listDocuments()

documents.js

Handles document ingestion for all supported file types. Contains a shared ingestText() core that chunks, embeds, and stores text, plus format-specific extractors:

Format	Library	Strategy
PDF	pdf-parse	Extract text from all pages
DOCX	mammoth	Extract raw text content
CSV	(built-in)	Convert rows to 'Header: value' natural language
TXT	(built-in)	Read as UTF-8 string directly

Exports: ingestDocument(buffer, filename), ingestPDF() [backward compat]

retriever.js

Embeds the user's query, searches ChromaDB for the top-K most similar chunks, and formats them into a context string for the LLM. The buildRagContext() function mirrors the existing buildSearchContext() pattern used by web search, with numbered citations and source filenames.

index.js

Orchestrator module that initializes embeddings and ChromaDB in parallel at startup, and re-exports all public functions as the module's API.

4. Server.js Modifications

Startup

The RAG module is initialized at server startup with a non-blocking call. If initialization fails (e.g., ChromaDB not ready yet), a warning is logged but the server continues to operate normally for non-RAG features.

Chat Endpoint (/api/chat)

The existing /api/chat endpoint was extended to support RAG alongside web search. When the request includes useRag: true, the server:

- Embeds the user's query and retrieves matching document chunks
- Formats the chunks into a context block injected into the system prompt
- Returns RAG source metadata alongside the LLM response

RAG and web search are mutually exclusive. If both are requested, web search takes precedence. The system prompt injection follows the same pattern: baseSystem + separator + context.

RAG Admin Endpoints

Method	Path	Purpose
POST	/api/rag/documents	Upload and ingest a document
GET	/api/rag/documents	List all ingested documents
DELETE	/api/rag/documents/:id	Delete a document and its chunks
GET	/rag-admin	Serve the admin page

File Upload Handling

Multer handles multipart file uploads with a 10MB size limit. The file filter accepts .pdf, .docx, .csv, and .txt extensions. Files are stored in memory (not disk) and passed directly to the ingestion pipeline.

5. Frontend Changes

chat-advanced.html

The existing RAG checkbox toggle was wired into the /api/chat request flow, following the same pattern as the web search toggle:

- RAG and Web Search checkboxes are mutually exclusive (checking one unchecks the other)
- When RAG is enabled, the thinking indicator shows 'Searching documents...'
- RAG sources display inline with snippet text (no URLs, unlike web search results)
- Debug panel shows RAG-specific metrics (ragUsed, ragResults count)

All existing functionality (streaming, memory, web search, theme toggle) continues to work unchanged.

rag-admin.html

A new dedicated page for document management, styled to match the existing dark/light theme system used across the webstack. Features:

- Drag-and-drop upload zone supporting PDF, DOCX, CSV, and TXT files
- Real-time processing status indicator during upload and ingestion
- Document list showing filename, upload date, chunk count, and character count
- Delete button with confirmation dialog for each document
- Dark/light theme toggle matching the chat pages
- Navigation link back to the chat interface

6. File Summary

Files Created

File	Purpose
app/rag/embeddings.js	Local embedding via @xenova/transformers
app/rag/vectorstore.js	ChromaDB client operations
app/rag/documents.js	Multi-format ingestion pipeline
app/rag/retriever.js	Query embedding + similarity search
app/rag/index.js	Module orchestrator and public API
app/public/rag-admin.html	Document upload and management UI

Files Modified

File	Change
docker-compose.yml	Added ChromaDB service, node:20-slim, volumes
Caddyfile	Added /rag-admin reverse proxy route
app/package.json	Added chromadb, transformers, mammoth, etc.
app/Server.js	RAG init, chat endpoint, admin API routes
app/public/chat-advanced.html	RAG toggle wiring, source rendering

7. Dependencies Added

Package	Purpose
chromadb	JavaScript client for ChromaDB vector database
@xenova/transformers	Run all-MiniLM-L6-v2 embedding model in Node.js
pdf-parse	Extract text content from PDF files
multer	Handle multipart file uploads in Express
mammoth	Extract text from DOCX (Word) files

8. Technical Challenges Solved

Alpine Linux vs glibc

The initial Docker image (node:20-alpine) caused crashes because @xenova/transformers depends on onnxruntime-node, which requires glibc. Alpine Linux uses musl instead. Resolved by switching to node:20-slim (Debian-based), which provides glibc.

ChromaDB Client Architecture

The ChromaDB JavaScript client is HTTP-only and cannot run as an in-process database (unlike the Python client). This required adding ChromaDB as a separate Docker service and connecting via the internal Docker network (<http://chromadb:8000>).

Embedding Function Requirements

ChromaDB's JS client demands an embedding function (chromadb-default-embed) when using document-based add/query operations. This package also pulled in onnxruntime and caused the same Alpine crashes. Resolved by computing embeddings independently via @xenova/transformers and passing pre-computed embedding vectors directly to ChromaDB.

LM Studio Embedding Limitations

An attempt to use LM Studio's /v1/embeddings endpoint failed because the GGUF-format embedding model could not be loaded by LM Studio's runtime. This confirmed that local embedding via @xenova/transformers was the correct approach.

9. Python RAG Application (Prototype)

Before integrating into the webstack, a standalone Python RAG application was built as a prototype at `/Users/ericfrayer/Development/rag_app_01/`. This served as the proof of concept and informed the architecture of the Node.js implementation.

Stack

- LangGraph StateGraph for the retrieve-then-generate pipeline
- LangChain ChatOpenAI class pointed at LM Studio (OpenAI-compatible API)
- ChromaDB (in-process persistent) for vector storage
- sentence-transformers / all-MiniLM-L6-v2 for embeddings
- FastAPI web interface for upload and chat

Key Files

File	Purpose
app/config.py	Central configuration (LM Studio URL, model, params)
app/ingest.py	PDF ingestion with incremental processing via manifest
app/graph.py	LangGraph RAG pipeline (retrieve -> generate)
app/main.py	CLI entry point for interactive queries
app/api.py	FastAPI web interface

The Python app features incremental ingestion: a manifest file (`ingested_manifest.json`) tracks which files have been processed and their modification times, so only new or changed files are re-ingested on subsequent runs.

10. Summary

This project successfully integrates a complete RAG pipeline into an existing production webstack, enabling document-grounded Q&A through the chat interface. The entire system runs locally with no external API dependencies for embeddings or vector storage.

Key outcomes:

- Four document formats supported: PDF, DOCX, CSV, and TXT
- Local embeddings via all-MiniLM-L6-v2 (no API calls, no data leaves the network)
- Persistent vector storage in ChromaDB survives container restarts
- Seamless integration with existing chat UI (toggle on/off, mutually exclusive with web search)
- Dedicated admin page for document lifecycle management
- All existing features preserved: streaming, memory, web search, themes

Document generated with Claude Code